

ROS TASK

Pull Task3 from [GitHub](#) in your existing directory on local system.

Note: For convenience, all the requisite files such as `CMakeLists.txt` and `package.xml` are already provided in the package directory. However, you are encouraged to create your own packages for curiosity, practice, and a better understanding of ROS 2 package structures.

Task 3A: Infinity Path Tracer

Aim

The objective of this task is to familiarize with ROS 2 publishers, velocity control, and trajectory generation using the Turtlesim simulator.

Problem Statement

Cyborg Robotics is developing an autonomous inspection robot that must navigate smoothly along predefined trajectories. Before deploying the control algorithms on a physical robot, the team wants to validate the motion behavior in simulation.

Your task is to develop a ROS 2 node that controls a turtle in the Turtlesim simulator and makes it trace an infinity-shaped (∞) trajectory.

The turtle must move continuously using velocity commands and complete at least one full infinity-shaped path without teleportation or manual repositioning.

Task Requirements

Your ROS 2 node must:

1. Control `turtle1` in the Turtlesim simulator.
2. Publish velocity commands to move the turtle.
3. Generate a smooth infinity-shaped trajectory.
4. Complete one full infinity loop.
5. Stop the turtle after completing the trajectory at the starting position.

Constraints

- Only velocity commands may be used to control the turtle.
- Teleport services are not allowed.
- Manual repositioning of the turtle is not allowed.
- The trajectory must be generated programmatically.
- The turtle should remain inside the simulator window during execution.

ROS Interfaces

Publisher

Topic:

`/turtle1/cmd_vel`

Message Type:

`geometry_msgs/msg/Twist`

Task 3A-Bonus: Archimedean Path Tracer

Problem Statement

Cyborg Robotics is developing an autonomous search robot for exploration missions. During search operations, the robot may be required to systematically scan an unknown environment by gradually expanding its search radius from a starting location.

Your task is to develop a ROS 2 node that controls a turtle in the Turtlesim simulator and makes it trace an **Archimedean Spiral** trajectory.

The turtle should begin near the center of the simulator window and continuously move outward while maintaining a smooth spiral path.

Task Requirements

1. Control turtle1 in the Turtlesim simulator.
2. Publish velocity commands using:
`/turtle1/cmd_vel`
3. Generate an Archimedean Spiral trajectory.
4. The radius of the spiral must continuously increase with time.
5. The turtle should remain within the simulator window for the duration of the execution.
6. Stop the turtle after completing the spiral.

Constraints

- Only velocity commands may be used.
 - Teleport services are not allowed.
 - Manual repositioning is not allowed.
 - The trajectory must be generated programmatically.
-

Task 3B: Waypoint navigation

Aim

The objective of this task is to familiarize with ROS 2 publishers, subscribers, feedback control, and waypoint-based navigation using the Turtlesim simulator.

Problem Statement

Cyborg Robotics is developing an autonomous inspection robot that must visit a sequence of predefined checkpoints during a facility inspection mission. Before deploying the robot in a real environment, the navigation algorithm must be validated in simulation.

Your task is to develop a ROS 2 node that autonomously drives a turtle through a sequence of waypoints. The turtle must continuously monitor its pose and navigate to each waypoint in the correct order.

Task Requirements

Your ROS 2 node must:

1. Subscribe to the turtle pose information.
2. Navigate through the following waypoints sequentially:

```
[5.5, 9.5],  
[7.0, 5.5],  
[10.0, 5.5],  
[7.8, 3.0],  
[9.0, 0.5],  
[5.5, 2.5],
```

```
[2.0, 0.5],  
[3.2, 3.0],  
[1.0, 5.5],  
[4.0, 5.5],  
[5.5, 9.5]
```

3. Switch to the next waypoint only after reaching the current waypoint.
4. Continuously compute position and heading error.
5. Generate velocity commands to reach each waypoint.
6. Stop the turtle after completing the entire path.

Constraints

- Only velocity commands may be used.
- Teleport services are not allowed.
- Manual repositioning is not allowed.
- The controller must use pose feedback from the simulator.

ROS Interfaces

Publisher

Topic:

`/turtle1/pose`

Message Type:

`/turtlesim/msg/Pose`

Subscriber

Topic:

`/turtle1/cmd_vel`

Message Type:

`/geometry_msgs/msg/Twist`

Task 3C: Police vs Thief Challenge

Problem Statement

In this task, you are required to develop a ROS 2 node that controls a police turtle to catch a moving thief turtle inside the turtlesim environment.

The thief turtle (**turtle1**) will continuously move to randomly generated locations within the arena. A second turtle named **police** will be spawned automatically. Your objective is to design a control algorithm that enables the police turtle to successfully pursue and capture the thief.

A capture is considered successful when the distance between the police turtle and the thief turtle becomes less than 0.5 units.

Objectives

1. Subscribe to the pose of the thief turtle.
2. Subscribe to the pose of the police turtle.
3. Generate appropriate velocity commands for the police turtle.
4. Successfully capture the thief turtle.
5. Print an appropriate message upon successful capture.

ROS Interfaces

Subscribed Topics

Topic	Message Type
<code>/turtle1/pose</code>	<code>turtlesim/msg/Pose</code>
<code>/police/pose</code>	<code>turtlesim/msg/Pose</code>

Published Topics

Topic	Message Type
<code>/police/cmd_vel</code>	<code>geometry_msgs/msg/Twist</code>

Constraints

- You must not modify the provided thief logic.
- You must not publish commands to `/turtle1/cmd_vel`.
- You must not use teleportation services.
- You must not reset or manipulate the turtlesim environment during execution.
- The solution must be implemented entirely in Python using ROS 2.

Expected Behaviour

- The thief turtle moves continuously inside the arena.
- The police turtle should autonomously pursue the thief.
- Upon capture, the police turtle should stop and display a capture message in the terminal.

Task3D : Service and Client

You must create the following:

1. Custom Service Definition

Design and implement a custom ROS 2 service that accepts a positive integer and returns all of its prime factors as an array.

2. Service Server Node

The server node shall:

- Host the custom service.
- Receive requests from the client.
- Compute the prime factorization of the given number.
- Return the prime factors as an array.

3. Service Client Node

The client node shall:

- Send a request containing a positive integer.
- Receive the response from the server.
- Display the returned prime factors in the terminal.

Note

You are required to design the complete service architecture yourself, including:

- Service definition (.srv)
- Service server implementation
- Service client implementation

Running Instructions

Step 1: Build the Workspace

Build your workspace using `colcon` build and source your terminal `source install/setup.bash`

Step 2: Launch the Environment

`ros2 launch task3 Task3.launch.py`

This launch file will start:

- Turtlesim node

Step 3: Run the Required Task

Examples:

```
ros2 run task3 Task3A.py
```

For Task 3D:

Start the server:

```
ros2 run task3 Task3D_server.py
```

Open a new terminal, source the workspace, and run the client:

```
ros2 run task3 Task3D_client.py
```

Submission Requirements

Before demonstrating your solution, you must start a screen recording.

Step 1: Start Screen Recording

The screen recording must begin before any commands are executed in the terminal and should clearly show:

- Opening the terminal.
- Running the bag recording command.
- Launching the environment.
- Executing the solution.
- The final output.

Step 2: Start Bag Recording

Before running any launch file or Python script, open a new terminal and execute:

```
ros2 bag record -a
```

Step 3: Execute the Task

Run the required commands as specified in the task instructions.

Step 4: Stop Bag Recording

After the task is completed, stop the bag recording using Ctrl + C

Rename the generated bag folder according to the task naming convention before submission.

Example:

```
<Rollno.>_Task3A.bag
```

Step 5: Take Screenshot

A screenshot of the successful execution/output must be captured and submitted.

The screenshot should clearly show:

- The turtlesim window (if applicable).
- Terminal output.
- Successful completion of the task.

Files to Submit

For each task, rename the required file as <Rollno.><Taskname>.<extension> like 124ec0035_TaskA.png / 124ec0035_TaskA.py and so on.

For each subtask, submit:

- Screen Recording (.mp4)
- Screenshot (.png)
- Generated Bag File
- Source Code (.py)

Ensure all files follow the prescribed naming convention and are uploaded to the correct subdirectory.

Deadline: 10 June EOD